



DAW

Generador de
documentación



Miruna Georgiana Matei

IES Juan Bosco

CONTENIDO

Investigación breve	2
JavaDoc	2
Facilidad de uso	2
Integración de proyectos.....	2
Formatos de salida	2
Comunidad	2
Curva de aprendizaje	3
Sphinx	3
Facilidad de uso	3
Integración de proyectos.....	3
Formatos de salida	4
Comunidad	4
Curva de aprendizaje	4
Doxygen	4
Facilidad de uso	4
Integración de proyectos.....	4
Formatos de salida	4
Comunidad	5
Curva de aprendizaje	5
Aplicación práctica	6
JavaDoc	6
Sphinx	7
Instalación de sphinx + Creación de fichero + HTML.....	7
Prueba definitiva.....	8
Doxygen	11
Conclusión	13
JavaDoc	13
Sphinx	13
Doxygen	14

INVESTIGACIÓN BREVE

JAVADOC

FACILIDAD DE USO

Es muy sencillo de usar porque se integra directamente en el código fuente mediante comentarios sencillos con un formato estándar. Se usan `/** ... */` justo encima de la clase, método o atributo que se quiera documentar.

Se usan etiquetas que empiecen por `@`

- **@param:** describe un parámetro que recibe un método
- **@return:** explica qué valor devuelve el método
- **@author:** indica el creador del código
- **@throws:** detalla las excepciones que puede lanzar el método

Además, IDEs como Eclipse o NetBeans, pueden autogenerar la estructura del comentario al escribir `/** ... */`, y permiten exportar toda la documentación a páginas web en formato HTML.

INTEGRACIÓN DE PROYECTOS

Con Javadoc podemos automatizar la generación de la documentación del código fuente de Java, usando Maven o Gradle.

- **Maven:** se gestiona mediante plugins oficiales, siendo el más común `maven-javadoc-plugin`
- **Gradle:** Es más sencillo porque incluye una tarea nativa llamada `JavaDoc` por defecto, sin necesidad de escribir todo manualmente.

FORMATOS DE SALIDA

Su formato de salida es el predeterminado y estándar de Javadoc, que es HTML, el cual genera un conjunto de páginas web navegables para consultar la API. Produce una estructura con índices, clases y métodos organizados. Además, permite usar programas externos llamados doclets para definir contenido y formatos alternativos y de ahí se pueden generar PDFs, XMLs, y otros formatos como SGML, RTF o MIF.

COMUNIDAD

Para su comunidad, JavaDoc es el estándar indiscutible para generar referencias de API. Todo programador de Java aprende a usarlo desde el principio para documentar clases, métodos, parámetros y valores mediante las etiquetas mencionadas anteriormente.

También, por Stack Overflow, Reddit o GitHub, suelen discutir la herramienta para resolver dudas prácticas como, por ejemplo, cómo configurar plugins avanzados en

herramientas como Maven o Gradle, o también realizan buenas prácticas sobre qué y qué no documentar para mantener un código limpio y legible.

CURVA DE APRENDIZAJE

Es extremadamente rápida y plana, prácticamente inexistente si ya tienes una base mínima de programación en Java. El nivel básico es aprenderse la sintaxis elemental de los comentarios usando la estructura `/** ... */` antes de clases o métodos. El nivel intermedio ya sería usando las etiquetas, y la más avanzada sería usando las herramientas de construcción como bien mencionamos antes.

SPHINX

FACILIDAD DE USO

Depende completamente del nivel de experiencia técnica y de lo que se intente documentar. No es una herramienta intuitiva, pero es el estándar de la industria para proyectos complejos de software.

Lo que se facilita a la hora de usarlo es el asistente de configuración inicial, que hace preguntas básicas y se autogenera la estructura de carpetas y archivos; la automatización de código, en Python puede leer los docstrings que ponemos y generar la documentación técnica automáticamente; comandos sencillos de compilación y se puede conectar con plataformas como **Read the Docs**, lo que automatiza la publicación en la nube cada vez que se suben cambios a GitHub.

INTEGRACIÓN DE PROYECTOS

Tiene la capacidad de unificar múltiples documentaciones independientes o de conectar Sphinx con otras herramientas y flujos de trabajo, como sistemas de control de versiones, pipelines de CI/CD o entornos de desarrollo.

Dependiendo de lo que busque cada uno lograr, hay 3 formas principales:

- **InterSphinx:** sirve para vincular documentaciones separadas y enlazarlos sin usar enlaces URLs fijos.
- **Autodoc:** sirve para integrar la documentación técnica directamente desde el código fuente, utiliza extensiones que extraen los docstrings.
- **Integración en Pipelines de CI/CD:** para que la documentación se actualice automáticamente con cada cambio en el proyecto, Sphinx se integra en los flujos de despliegue continua, usando **Read the Docs**, que es la plataforma más común que podemos conectar nuestro repositorio de GitHub y cada vez que hacemos un git push, el **Read the Docs** compila e integra el proyecto Sphinx en la web.

FORMATOS DE SALIDA

Puede ser Web y documentación interactiva: HTML, Single HTML (genera todo el contenido de la documentación en una sola página HTML) y JSON; formatos de lectura e impresión: PDF a través de LaTeX y EPUB (genera un archivo .epub permitiendo exportar toda la documentación técnica para leerla cómodamente en libros electrónicos y tablets); formatos de texto y manuales de sistema: man pages, texinfo (para compilar manuales del proyecto GNU) y plain text.

COMUNIDAD

La comunidad utiliza de normal Read the Docs ya que es la plataforma de alojamiento por excelencia. La gran mayoría de proyectos usan Sphinx ya que automatizan la publicación de su documentación de forma gratuita, actualizándose con cada cambio en el repositorio. También cuenta con otras extensiones como Autodoc, extrayendo los comentarios y las explicaciones directamente de los bloques de código para generar la referencia de la API sin esfuerzo manual y, por último, hay una amplia variedad de plantillas estéticas y responsivas para mejorar la experiencia de lectura.

CURVA DE APRENDIZAJE

Es de moderada a alta, se sitúa por encima de las alternativas simples como MkDocs debido a su potencia, extensibilidad y dependencias técnicas. Aunque la inicialización es rápida, dominar la jerarquía de contenidos y la automatización de código exige familiarizarse con conceptos avanzados.

DOXYGEN

FACILIDAD DE USO

Extrae los comentarios del código, hace que las bases de código sean claras para colaboradores, maneja pequeños scripts o proyectos masivos de forma sencilla y su documentación puede impresionar a clientes o profesores.

INTEGRACIÓN DE PROYECTOS

Automatiza la generación de manuales técnicos, diagramas y referencias a partir de los comentarios del código fuente como C++, C, Java, Python y VHDL.

FORMATOS DE SALIDA

- **HTML:** páginas web interactivas con índices, árboles de navegación y diagramas. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_HTML = YES`
- **LaTeX / PDF:** genera código fuente en LaTeX que permite compilar documentos listos para impresión o conversión a PDF. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_LATEX = YES`

- **XML:** estructura de datos completa en XML para ser procesada por otras herramientas. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_XML= YES`
- **Man pages:** páginas de manual estándar para sistemas operativos como Linux y tipo Unix. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_MAN= YES`
- **RTF:** formato de texto enriquecido optimizado para su importación en Word. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_RTF= YES`
- **DocBook:** formato basado en XML para documentación técnica. Y de forma que queremos activarlo, simplemente hay que poner: `GENERATE_DOCBOOK= YES`

COMUNIDAD

Es técnica, madura, colaborativa y muy orientada al desarrollo de software de código abierto. Son perfiles técnicos y profesionales, se centran en la eficiencia, automatización y compatibilidad con sistema CI/CD. Se creó en el año 1997, por lo que la comunidad está muy establecida; los usuarios valoran que la herramienta siga siendo fiable y compatible con versiones anteriores.

CURVA DE APRENDIZAJE

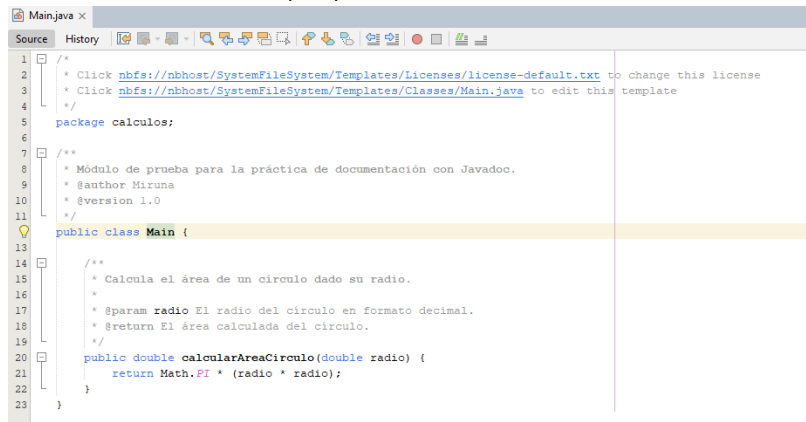
Como principiante se puede usar **Doxywizard**, el asistente visual, para rellenar los datos del proyecto mediante formularios. Luego, para documentar funciones, clases y variables hay que saberle algunas etiquetas. Y de forma más avanzada se pueden configurar archivos complejos con herramientas externas como **Graphviz** para generar árboles de herencia y diagramas avanzados.

Conclusión: Es moderado al principio, pero se vuelve muy rápido y sencillo si se utiliza su interfaz gráfica y siguiendo una estructura básica de comentarios.

APLICACIÓN PRÁCTICA

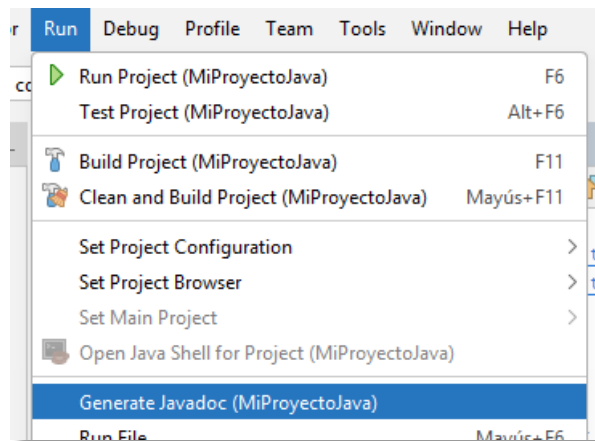
JAVADOC

1. Creamos nuestro proyecto Java



```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
4  */
5  package calculos;
6
7  /**
8   * Módulo de prueba para la práctica de documentación con Javadoc.
9   * @author Miruna
10  * @version 1.0
11  */
12
13  public class Main {
14
15      /**
16       * Calcula el área de un círculo dado su radio.
17       *
18       * @param radio El radio del círculo en formato decimal.
19       * @return El área calculada del círculo.
20       */
21      public double calcularAreaCirculo(double radio) {
22          return Math.PI * (radio * radio);
23      }
24  }
```

2. Generamos nuestro Javadoc. Arriba a la izquierda en Run seleccionamos Generate Javadoc



3. Resultado HTML



INSTALACIÓN DE SPHINX + CREACIÓN DE FICHERO + HTML

4. Crear un entorno virtual

```
C:\Users\Usuario\Desktop>cd pruebaPython  
C:\Users\Usuario\Desktop\pruebaPython>python -m venv venv  
C:\Users\Usuario\Desktop\pruebaPython>|
```

5. Lo activamos e instalamos sphinx

```
C:\Users\Usuario\Desktop\pruebaPython>venv\Scripts\activate.bat  
(venv) C:\Users\Usuario\Desktop\pruebaPython>pip install Sphinx  
Collecting Sphinx  
  Downloading sphinx-9.1.0-py3-none-any.whl.metadata (5.8 kB)  
Collecting sphinxcontrib-applehelp>=1.0.7 (from Sphinx)  
  Downloading sphinxcontrib_applehelp-2.0.0-py3-none-any.whl.metadata (2.3 kB)  
Collecting sphinxcontrib-devhelp>=1.0.6 (from Sphinx)  
  Downloading sphinxcontrib_devhelp-2.0.0-py3-none-any.whl.metadata (2.3 kB)
```

6. Creamos una carpeta llamada docs y accedemos para iniciar

```
(venv) C:\Users\Usuario\Desktop\pruebaPython>mkdir docs  
(venv) C:\Users\Usuario\Desktop\pruebaPython>cd docs  
(venv) C:\Users\Usuario\Desktop\pruebaPython\docs>sphinx-quickstart  
Welcome to the Sphinx 9.1.0 quickstart utility.
```

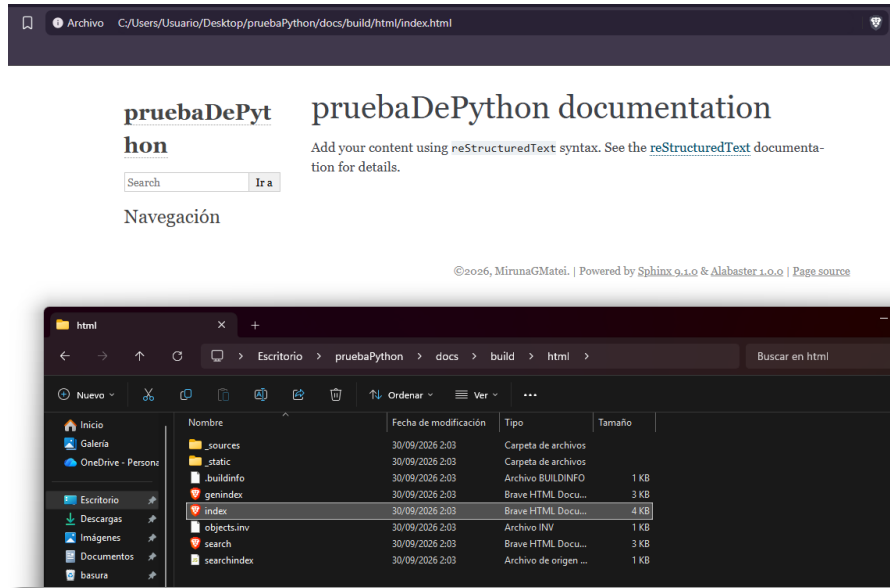
7. Rellenamos los datos

```
The project name will occur in several places in the built documentation.  
> Project name: pruebaDePython  
> Author name(s): MirunaGMatei  
> Project release []:  
  
If the documents are to be written in a language other than English,  
you can select a language here by its language code. Sphinx will then  
translate text that it generates into that language.  
  
For a list of supported codes, see  
https://www.sphinx-doc.org/en/master/usage/configuration.html#confval-language.  
> Project language [en]: es
```

8. Creamos el fichero HTML

```
(venv) C:\Users\Usuario\Desktop\pruebaPython\docs>make html  
Running Sphinx v9.1.0  
loading translations [es]... done  
making output directory... done  
building [mo]: targets for 0 po files that are out of date  
writing output...  
building [html]: targets for 1 source files that are out of date  
updating environment: [new config] 1 added, 0 changed, 0 removed  
reading sources... [100%] index  
looking for now-outdated files... none found
```

9. Resultado



PRUEBA DEFINITIVA

1. Crear y activar el entorno virtual en caso de que no esté creado

```
C:\Users\Usuario\Desktop\MiProyectoPython>venv\Scripts\activate
(venv) C:\Users\Usuario\Desktop\MiProyectoPython>
```

2. Instalamos sphinx y una extensión para leer Markdown

```
(venv) C:\Users\Usuario\Desktop\MiProyectoPython>pip install Sphinx myst-parser
Collecting Sphinx
  Using cached sphinx-9.1.0-py3-none-any.whl.metadata (5.8 kB)
Collecting myst-parser
  Using cached myst-parser-5.1.0-py3-none-any.whl.metadata (5.6 kB)
```

3. Añadimos comentarios de documentación en el código

```
mi_codigo.py
C:\Users\Usuario\Desktop\MiProyectoPython>mi_codigo.py
1 """Módulo de prueba para la documentación con Sphinx."""
2
3 def saludar(nombre: str) -> str:
4     """Devuelve un saludo personalizado.
5
6     Args:
7         nombre (str): Nombre de la persona a saludar.
8
9     Returns:
10         str: Mensaje de saludo.
11     """
12     return f"Hola, {nombre}!"
```

4. Creamos la carpeta docs e instalamos el soporte para Markdown, para inicializar la documentación

```
(venv) C:\Users\Usuario\Desktop\MiProyectoPython\docs>python -m sphinx.cmd.quickstart
Welcome to the Sphinx 9.1.0 quickstart utility.
```

5. Rellenamos los datos

```
The project name will occur in several places in the built documentation.
> Project name: MiProyectoPython_Documentacion
> Author name(s): MirunaGMatei
> Project release []: 1.0

If the documents are to be written in a language other than English,
you can select a language here by its language code. Sphinx will then
translate text that it generates into that language.

For a list of supported codes, see
https://www.sphinx-doc.org/en/master/usage/configuration.html#confval-language.
> Project language [en]: es
```

6. Resultado Markdown Se encuentra dentro de docs/source/

```
index.rst X
C:\Users\Usuario\Desktop\MiProyectoPython> docs > source > index.rst
1  .. MiProyectoPython_Documentacion documentation master file, created by
2     sphinx-quickstart on Wed Sep 30 02:25:24 2026.
3     You can adapt this file completely to your liking, but it should at least
4     contain the root `toctree` directive.
5
6     MiProyectoPython_Documentacion documentation
7     =====
8
9     Add your content using `reStructuredText` syntax. See the
10    `reStructuredText` <https://www.sphinx-doc.org/en/master/usage/restructuredtext/index.html> _
11    documentation for details.
12
13
14    .. toctree::
15       :maxdepth: 2
16       :caption: Contents:
17
18
```

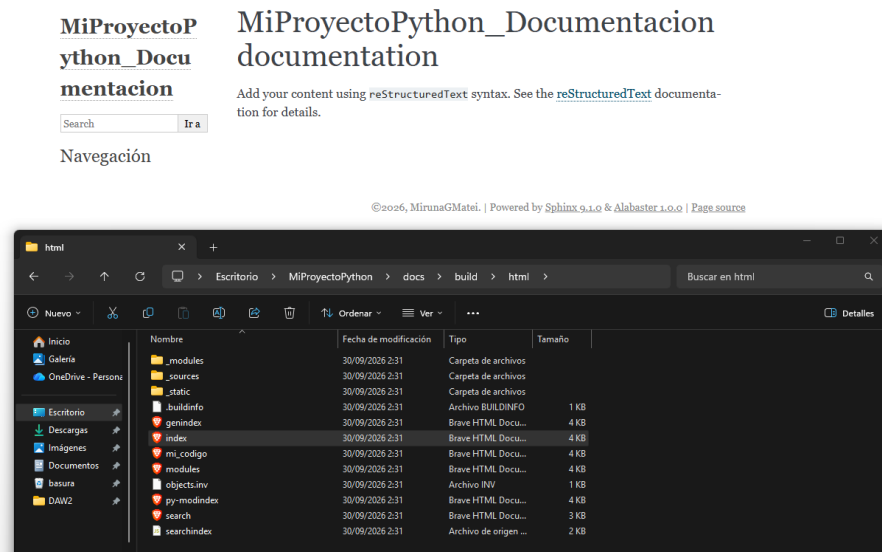
7. Modificar conf.py de source para poder crear una documentación HTML

```
conf.py X
C:\Users\Usuario\Desktop\MiProyectoPython> docs > source > conf.py > ...
1  import os
2  import sys
3  sys.path.insert(0, os.path.abspath('../..'))
4
5  # Configuration file for the Sphinx documentation builder.
6  #
7  # For the full list of built-in configuration values, see the documentation:
8  # https://www.sphinx-doc.org/en/master/usage/configuration.html
9
10 # -- Project information -----
11 # https://www.sphinx-doc.org/en/master/usage/configuration.html#project-information
12
13 project = 'MiProyectoPython_Documentacion'
14 copyright = '2026, MirunaGMatei'
15 author = 'MirunaGMatei'
16 release = '1.0'
17
18 # -- General configuration -----
19 # https://www.sphinx-doc.org/en/master/usage/configuration.html#general-configuration
20
21 extensions = [
22     'sphinx.ext.autodoc',
23     'sphinx.ext.viewcode',
24 ]
25
26 templates_path = ['_templates']
27 exclude_patterns = []
28
29 language = 'es'
30
31 # -- Options for HTML output -----
32 # https://www.sphinx-doc.org/en/master/usage/configuration.html#options-for-html-output
33
34 html_theme = 'alabaster'
35 html_static_path = ['_static']
36
```

Se añaden para poder generar el documento de forma automática

```
(venv) C:\Users\Usuario\Desktop\MiProyectoPython\docs> sphinx-apidoc -o . ../..
(venv) C:\Users\Usuario\Desktop\MiProyectoPython\docs\source> cd ..
(venv) C:\Users\Usuario\Desktop\MiProyectoPython\docs> make html
Running Sphinx v9.1.0
```

7.1. Resultado HTML



8. Modificar conf.py para añadir para que nos de el formato PDF

```
extensions = [  
    'sphinx.ext.autodoc',  
    'sphinx.ext.viewcode',  
    'myst_parser',  
]
```

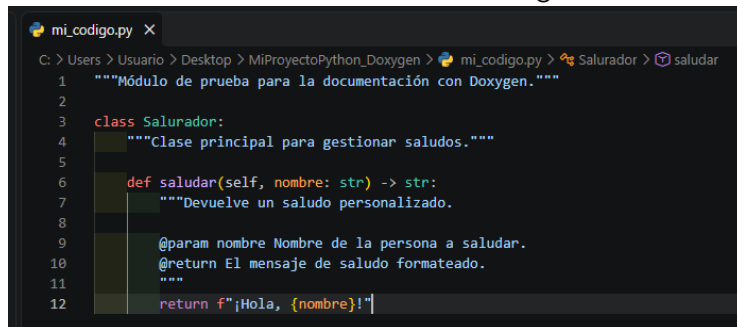
8.1. Sacar documentación formato PDF

```
C:\Users\Usuario\Desktop\MiProyectoPython>venv\Scripts\activate.bat  
(venv) C:\Users\Usuario\Desktop\MiProyectoPython>cd docs  
(venv) C:\Users\Usuario\Desktop\MiProyectoPython\docs>make latexpdf  
Running Sphinx v9.1.0
```

Y ya se tendría que haber creado, pero Windows no incluye de serie un motor de LaTeX como pdflatex, por lo que el comando make se queda a medias al no encontrar el programa encargado de transformar esos archivos de texto en un PDF real.

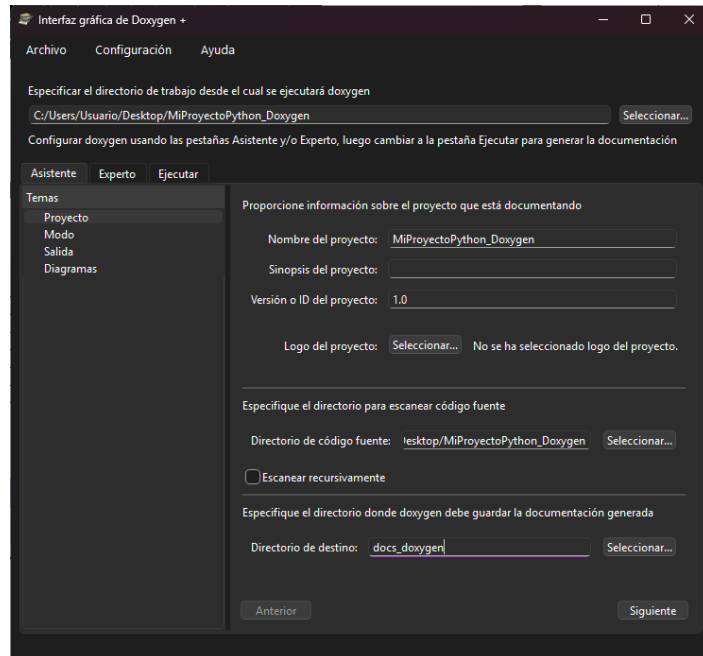
DOXYGEN

1. Instalamos Doxygen, que vendrá con Doxywizard para facilitar el relleno de datos sin tocar archivos de configuración complejos
2. Añadimos comentarios en nuestro código



```
mi_codigo.py X
C:\Users\Usuario\Desktop\MiProyectoPython_Doxygen > mi_codigo.py > Salurador > saludar
1  """Módulo de prueba para la documentación con Doxygen."""
2
3  class Salurador:
4      """Clase principal para gestionar saludos."""
5
6      def saludar(self, nombre: str) -> str:
7          """Devuelve un saludo personalizado.
8
9          @param nombre Nombre de la persona a saludar.
10         @return El mensaje de saludo formateado.
11         """
12         return f"¡Hola, {nombre}!"
```

3. Abrimos Doxywizard y rellenamos los datos necesarios



Interfaz gráfica de Doxygen +

Archivo Configuración Ayuda

Especificar el directorio de trabajo desde el cual se ejecutará doxygen
C:/Users/Usuario/Desktop/MiProyectoPython_Doxygen [Seleccionar...]

Configurar doxygen usando las pestañas Asistente y/o Experto, luego cambiar a la pestaña Ejecutar para generar la documentación

Asistente Experto Ejecutar

Temas

- Proyecto
- Modo
- Salida
- Diagramas

Proporcione información sobre el proyecto que está documentando

Nombre del proyecto: MiProyectoPython_Doxygen

Síntesis del proyecto:

Versión o ID del proyecto: 1.0

Logo del proyecto: [Seleccionar...] No se ha seleccionado logo del proyecto.

Especifique el directorio para escanear código fuente

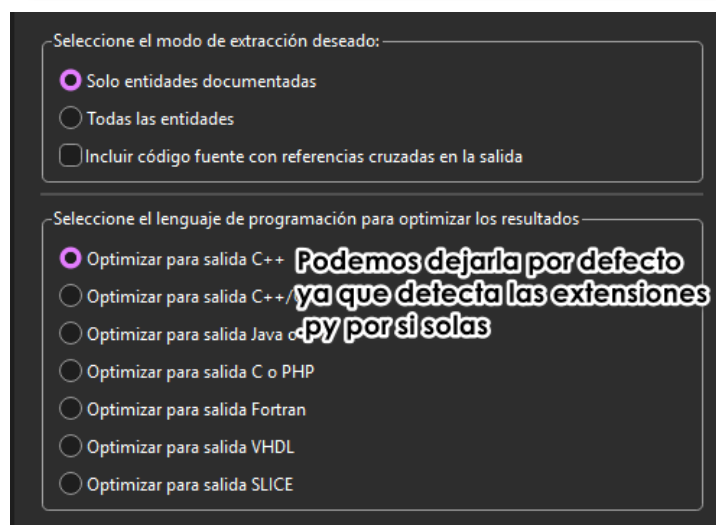
Directorio de código fuente: desktop/MiProyectoPython_Doxygen [Seleccionar...]

☐ Escanear recursivamente

Especifique el directorio donde doxygen debe guardar la documentación generada

Directorio de destino: docs_doxygen [Seleccionar...]

[Anterior] [Siguiente]



Seleccione el modo de extracción deseado:

☒ Solo entidades documentadas

☐ Todas las entidades

☐ Incluir código fuente con referencias cruzadas en la salida

Seleccione el lenguaje de programación para optimizar los resultados

☒ Optimizar para salida C++

☐ Optimizar para salida C++/Python

☐ Optimizar para salida Java

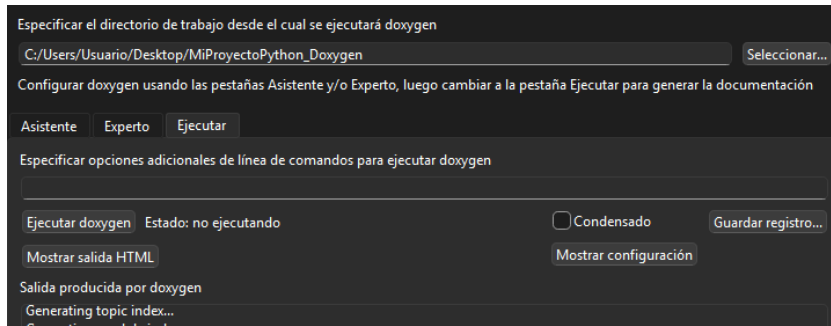
☐ Optimizar para salida C o PHP

☐ Optimizar para salida Fortran

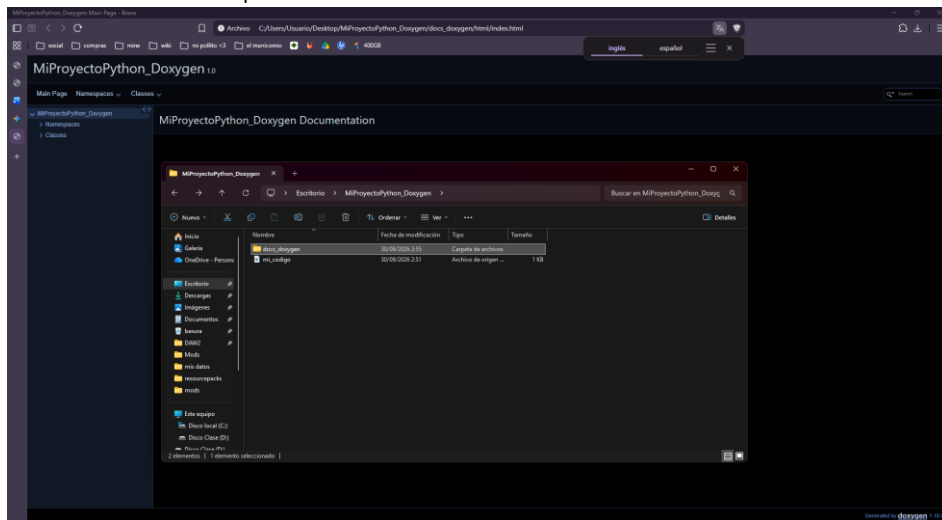
☐ Optimizar para salida VHDL

☐ Optimizar para salida SLICE

Podemos dejarla por defecto ya que detecta las extensiones .py por sí solas



4. Mostramos por HTML y PDF, pero de nuevo, no tenemos para el motor del PDF, pero aún así aparecerá su carpeta y la del HTML. Doxywizard no tiene para Markdown.



CONCLUSIÓN

JAVADOC

¿Qué tan fácil fue usar la herramienta?

Ha sido bastante sencillo, porque es simplemente poner los comentarios con sus respectivas etiquetas, y explicar con nuestras palabras de que se trata cada cosa y con darle a que genere un javadoc ya lo tenemos.

¿Qué ventajas / desventajas encontraste?

¿Ventajas? Sencillo de usar, aunque algo poco intuitivo porque no recordaba que estuviese en el apartado Run, pero muy sencillo y página intuitiva para entender y saber usar JavaDoc

¿Desventajas? No tiene para formato PDF o Markdown, pero con hacer ctrl + p para descargarlo formato PDF o hacer a mano el Markdown tampoco supondría un gran problema.

¿La recomendarías para un proyecto colaborativo?

Sinceramente sí, como ya he dicho es sencillo, útil y no tienes mucho estrés para saber si lo hiciste correctamente o no.

SPHINX

¿Qué tan fácil fue usar la herramienta?

Sinceramente, para mi fue bastante compleja. He tenido que repetirlo varias veces porque no entendía si iba por un buen camino, no sé si he realizado algún paso mal o he confundido algunos apartados.

¿Qué ventajas / desventajas encontraste?

¿Ventajas? Es excelente para automatizar de docstring en Python

¿Desventajas? Es complicado, necesitas un alto conocimiento para configurar manualmente las rutas, extensiones y comprender la sintaxis y necesita dependencia externa como los motores de LaTeX.

¿La recomendarías para un proyecto colaborativo?

Sinceramente, depende. Para los compañeros de clase no lo recomendaría porque no todos tenemos el mismo conocimiento y hay algunos que están más interesados en otros temas que en este tipo de documentación. Posiblemente, en caso de que estén interesados, les recomendaría otro generador de documentación, pero definitivamente no recomendaría Sphinx.

¿Qué tan fácil fue usar la herramienta?

Es sencillo de usar, la verdad, es también bastante cómodo porque no hay que poner ningún comando como tal, simplemente los datos y ajustarlo a tus condiciones. Además de que es bastante sencillo por si quiero hacerlo por HTML y PDF, me las hace a la vez y no tengo que ir una a una.

¿Qué ventajas / desventajas encontraste?

¿Ventajas? Tiene múltiples formatos de salida y soporta varios lenguajes de programación. Además, permite crear diagramas de clases, estructuras y árboles de herencia.

¿Desventajas? Las interfaces HTML generadas por defecto tienen un aspecto muy oscuro a mi parecer. También, aunque sea interfaz gráfica, puede resultar abrumador para principiantes.

¿La recomendarías para un proyecto colaborativo?

Posiblemente sí, pero ya a personas más especializadas y/o tengan un conocimiento básico sobre Doxygen porque como bien dije antes, es cómodo, pero un poco abrumador.